

Practitioner Craft — Numerics, Async, and the Idioms That Bite

Track 16.1 – 🐍 Python for AI Engineering · 16-ai-tech-stack/tracks/python/notes/02-numerics-async-and-performance.md

Scope: making Python fast enough, concurrent enough, and correct enough for production AI work. Toolchain, typing and testing are in 01-modern-python-toolchain.md.

1. The one number to internalise

A Python-level operation costs roughly **50–100 nanoseconds** of interpreter overhead. A float add in C costs under a nanosecond. So any loop doing simple arithmetic over more than a few thousand elements is spending 99% of its time on bookkeeping: bytecode dispatch, reference counting, boxing every float into a PyObject.

`code/vectorization_benchmark.py` [16-ai-tech-stack/tracks/python/code/vectorization_benchmark.py] measures this on pairwise distances — the inner loop of k-NN and therefore of vector retrieval. A representative run:

Approach	Time	vs. baseline
Pure Python triple loop	1.12 s	1×
NumPy <i>inside</i> a Python loop	214 ms	5×
Broadcasting	15.6 ms	72×
BLAS gemm identity	0.34 ms	3295×

The second row is the instructive one. People see `numpy` in the code and believe it is vectorized. It is not: it is 120,000 separate dispatches into C, each paying a few microseconds of fixed overhead. **Importing NumPy does not make code fast; removing the Python loop does.**

The last row is a different lesson: $||a-b||^2 = ||a||^2 - 2a \cdot b + ||b||^2$ turns the whole computation into one matrix multiply, which BLAS executes with multi-threaded, cache-blocked, SIMD assembly. Every vector database and every `sklearn` distance function does exactly this rewrite.

2. Broadcasting: the rules, and the bill

Broadcasting aligns shapes **from the right**. Two dimensions are compatible if they are equal, or one of them is 1. Missing leading dimensions are treated as 1.

```
(300, 1, 128)      A[:, None, :]
( 1, 400, 128)     B[None, :, :]
-----
(300, 400, 128)    result - 15.4M elements materialised in memory
```

That last line is the part benchmarks hide. In float64 that intermediate is **122.9 MB** for a final answer of 0.96 MB — a 128× overhead. Scale to 20,000 × 20,000 embeddings and it is 409 GB. Same line of code, different outcome: the OOM killer.

Rule: every time broadcasting adds a dimension, compute what that dimension costs in bytes. If the answer is uncomfortable, batch the outer loop (chunk 1,000 rows at a time) or find the algebraic identity that avoids the intermediate.

`dtype` is the other lever nobody pulls. NumPy defaults to float64; every embedding model, GPU tensor core and vector index in your stack is float32 or narrower. Storing embeddings as float64 doubles your memory bill and halves your bandwidth for precision the model never had. Cast at the boundary: `np.asarray(x, dtype=np.float32)`.

3. The silent broadcasting bug

This is the one that ships:

```
x = emb - emb.mean(axis=1)    # meant axis=0. Column means, not row means.
```

With a (5, 3) array this raises `ValueError` — a gift. With a **square** batch, the shapes align by accident, NumPy computes something perfectly well-typed and entirely wrong, and nothing warns you. Your loss still goes down. Your eval is quietly 4% worse. You spend a week blaming the learning rate.

Three habits that cost nothing:

- **keepdims=True** on every reduction you will broadcast against. `mean(axis=1, keepdims=True)` has shape `(n, 1)`, which can only align one way. The rank-1 version can align two ways, and will pick the wrong one.
- **Assert shapes at function boundaries.** `assert emb.shape == (n, d)`, `emb.shape` is one line and it has saved more debugging hours than any profiler.
- **Never smoke-test with a square batch.** A 32×32 test tensor hides every axis bug you have.

Run §3 of the benchmark script to watch it happen.

4. Dataframes: pandas, polars, and the honest comparison

	pandas	polars
Engine	NumPy/Arrow, single-threaded	Rust, Arrow-native, multi-threaded
Execution	eager	eager and lazy (query optimiser, predicate pushdown)
Larger-than-memory	no	yes, streaming engine
API	~20 years of accumulated surface	smaller, stricter, more consistent
Ecosystem	everything integrates with it	good and growing, not universal
Typical speedup	1×	3–30× on group-by/join-heavy work
NaN vs null	conflated, historically painful	separate concepts, correctly

What I'd pick: polars for new data pipelines, pandas when you are handed a notebook or need a library that only speaks `DataFrame`. `df.to_pandas()` / `pl.from_pandas()` are cheap when both are Arrow-backed, so this is not a religious commitment — you can use polars for the heavy transform and hand pandas the result.

The idiom that converts pandas users:

```
import polars as pl

(pl.scan_parquet("logs/*.parquet")      # LAZY. Nothing is read yet.
 .filter(pl.col("model") == "claude-sonnet-4-5")
 .group_by("prompt_version")
 .agg(pl.col("latency_ms").median(), pl.len().alias("n"))
 .sort("n", descending=True)
 .collect())                          # NOW it runs, reading only needed columns and row groups
```

The optimiser pushes the filter and column selection into the Parquet reader, so you never materialise the columns you did not ask for. That is where most of the speedup comes from — not raw Rust, but not doing work.

When to leave dataframes entirely: if the data fits in memory and the operation is uniform numeric, plain NumPy is faster than either. If it does not fit and the query is SQL-shaped, use **DuckDB** — it reads Parquet directly, runs real SQL, spills to disk, and interoperates with both dataframes in zero copies.

5. When to leave Python

Path	Effort	Payoff	Use when
Vectorize with NumPy	minutes	10–1000×	the operation is uniform over an array
<code>functools.cache</code>	seconds	∞ on repeats	pure function, repeated arguments
Numba <code>@njit</code>	~1 hour	10–100×	numeric loops that will not vectorize
Cython	~1 day	10–100×	you need fine control and C interop
Rust + PyO3	~1 week	10–1000×	it is a library others will depend on
<code>torch.compile</code>	~1 hour	1.3–3×	the code is already tensors
Multiprocessing	~1 hour	~n_cores	embarrassingly parallel, CPU-bound
asyncio	~1 hour	10–100×	IO-bound — which LLM work always is

Note the modern Python stack made its own choice here: `tokenizers`, `polars`, `ruff`, `uv` and `pydantic-core` are all Rust behind a Python API. You get the ergonomics and someone else pays the performance tax.

The decision procedure, in order: **profile first**, then vectorize, then check whether you are actually IO-bound (you usually are), and only then compile. The failure mode is inverting this — three days rewriting a function in Rust that accounted for 4% of runtime.

6. Concurrency vs parallelism, and the GIL

They are different problems:

- **Concurrency** = dealing with many things at once (structure). One cook, many pots, nothing burning.
- **Parallelism** = doing many things at once (execution). Four cooks.

The **GIL** (Global Interpreter Lock) means only one thread executes Python bytecode at a time. Consequences people get wrong in both directions:

- Threads do **not** speed up pure-Python CPU work. That part is true.
- Threads **do** help when the work releases the GIL — and NumPy, PyTorch, `hashlib`, compression, and every blocking IO call all release it. A thread pool around `np.linalg` calls genuinely parallelises.
- The GIL is irrelevant to LLM workloads. You are blocked on a socket 80 ms away, not on bytecode.

Free-threaded CPython, as of 2026: PEP 703 made a no-GIL build official. It shipped as an *experimental* build in 3.13; in 3.14 the free-threaded build became a **supported, non-experimental** option — but it is still not the default interpreter, and C extensions must be built for it. Practical advice: assume the GIL, check `sys._is_gil_enabled()` if you must branch, and benchmark the free-threaded build for CPU-heavy data preparation. Do not default your service to it, and do not reach for it to speed up API calls — that is the wrong tool for a problem asyncio already solved.

Workload	Tool	Why
LLM / HTTP / DB calls	<code>asyncio</code>	thousands of concurrent waits, one thread
Blocking library you cannot make async	<code>asyncio.to_thread</code>	keeps the loop free
NumPy / torch compute	threads	the GIL is released during the kernel
Pure-Python CPU (tokenizing, chunking)	<code>ProcessPoolExecutor</code>	true parallelism, pays a pickling tax
Cross-machine	a queue (Celery, RQ, SQS)	it stopped being a Python problem

7. Async for LLM workloads

An LLM call is ~99.9% waiting. Sequential batching gives you throughput of $1/\text{latency}$ forever, regardless of your hardware. `code/async_llm_client.py` [16-ai-tech-stack/tracks/python/code/async_llm_client.py] demonstrates the whole pattern offline; a representative run shows **7.25 s sequential** → **0.76 s gathered** for 24 prompts.

```
# The most expensive line of Python in the LLM ecosystem:
results = [await client.complete(p) for p in prompts]          # n * latency

# The fix:
results = await asyncio.gather(*(client.complete(p) for p in prompts))  # ~max(latency)
```

Four rules on top of that:

Bound the concurrency. `gather` over 5,000 prompts opens 5,000 sockets and earns 5,000 429s. An `asyncio.Semaphore` is four lines. Size it with Little's Law: $\text{max_concurrency} \approx \text{target_RPS} \times \text{avg_latency_s}$. Thirty requests/second at 2 s latency needs ~60 in flight, not 5,000. Then set it ~20% under the provider's documented cap.

`return_exceptions=True`. Without it, one failure discards every other result in the batch — results you already paid for in tokens. With it, failures arrive as values you can inspect and retry selectively.

Retry with exponential backoff and jitter. The jitter is the part people drop, and it is the part that matters. If 500 workers are rate-limited at the same instant and all sleep exactly 1.0 s, they all wake at the same instant and hit the provider again — a thundering herd that turns a blip into an outage. Use full jitter: `sleep = random.uniform(0, min(cap, base * 2**attempt))`.

Retry the right errors. 429 and 5xx are transient. A 400 will be a 400 forever; retrying it just burns your budget more slowly. And respect `Retry-After` when the provider sends it — that header is the answer, your formula is a guess.

The traps

- **Never block the event loop.** `time.sleep`, `requests.get`, a tight numeric loop, `json.loads` on a 50 MB file — any of these freeze every other coroutine. The demo script measures it: 8×0.15 s of awaiting takes

0.15 s; 8×0.15 s of CPU spinning takes 1.20 s. Wrap blocking work in `asyncio.to_thread`. Enable `PYTHONASYNCIODEBUG=1` in development and it will warn you.

- **One `httpx.AsyncClient` per process, not per request.** A fresh client throws away connection pooling and TLS session reuse — 30–80 ms of handshake per call.
- **Set a read timeout that fits your longest generation.** A 90-second completion is not a hung socket, but the default timeout thinks it is.
- **Check your SDK's built-in retries before adding your own.** The `anthropic` and `openai` clients already back off with jitter. Stacked retries multiply: $3 \times 3 = 9$ calls for one logical request.
- **Prefer `asyncio.TaskGroup` (3.11+) to bare `create_task`.** A task you create and never await is a task whose exception disappears into a log line nobody reads. `TaskGroup` cancels siblings on failure and raises an `ExceptionGroup`.

8. Memory, generators, and profiling

Generators: the laziest useful idea in Python

```
rows = [parse(line) for line in open("corpus.jsonl")]    # entire file in RAM
rows = (parse(line) for line in open("corpus.jsonl"))    # one line at a time
```

One character, unbounded memory difference. For a 40 GB corpus that is the difference between a pipeline and an OOM. The cost: a generator is single-pass and has no `len()`. Be deliberate about which you want.

`itertools` is the standard library at its best. Four that pull their weight in AI code:

```
from itertools import batched, chain, islice, groupby

for chunk in batched(documents, 64):    # 3.12+. Batch for an embedding API.
    await embed(chunk)
islice(stream, 100)                    # first 100 without materialising the rest
chain.from_iterable(list_of_lists)     # flatten lazily
```

Caching

```
from functools import cache, lru_cache

@cache                                # unbounded – only for a bounded key space
def token_count(text: str) -> int: ...

@lru_cache(maxsize=10_000)             # bounded – the safe default
def embed_query(text: str) -> tuple[float, ...]: ...
```

Two gotchas. Arguments must be hashable, so a cached function cannot take a list or dict — return tuples, not lists, from cached functions. And `@cache` on a method keeps `self` alive forever, leaking every instance you ever cached against; use an explicit dict or `cachetools` for instance-scoped caches.

Provider-side **prompt caching** is a different and much bigger lever for LLM cost: put the stable prefix (system prompt, tool definitions, long document) first and the variable part last, so the cache can hit. That is Module

04/11 material, but it starts with how you order strings in Python.

Profiling

```
python -m cProfile -s cumtime app.py | head -40      # deterministic, ~2x overhead, function-level
uv run py-spy top --pid 12345                       # sampling, attaches to a RUNNING process
uv run py-spy record -o flame.svg -- python app.py   # flame graph, ~0 overhead
uv run python -m memray run app.py                  # who allocated what, line by line
python -X importtime app.py 2>&1 | sort -k2 -n       # 4-second CLI startup? it's an import
```

`py-spy` is the one that matters in production: it attaches to a live process without restarting it or importing anything into it. When a service is mysteriously slow at 3am, `py-spy dump --pid` gives you every thread's stack in one second.

Three rules: **measure before optimising** (you are wrong about where the time goes — always); **measure the right thing** (`time.perf_counter`, best-of-N, not mean, and never `time.time`); **check memory too**, because the fastest code that OOMs is not fast.

9. Python idioms that bite AI engineers

9.1 Mutable default arguments

```
def add_message(msg: str, history: list[str] = []) -> list[str]:    # WRONG
    history.append(msg)
    return history
```

The default is evaluated **once, at function definition**. Every call without `history` shares one list — so conversation #2 inherits conversation #1's messages. In an LLM app this manifests as context bleeding between users, which is a privacy incident, not a bug. Fix: `history: list[str] | None = None`, then `history = []` if `history` is `None` else `history`. Ruff's `B006` catches it; enable bugbear.

9.2 Closure late binding

```
tasks = [lambda: call(p) for p in prompts]    # every lambda calls with the LAST prompt
tasks = [lambda p=p: call(p) for p in prompts] # bind at definition time – fixed
```

Closures capture the *variable*, not its value at creation. This bites hardest when building lists of deferred LLM calls or retry callbacks — you fire 100 requests and get 100 copies of the same one.

9.3 Float equality

```
0.1 + 0.2 == 0.3      # False
```

Never `==` on floats. Use `math.isclose(a, b, rel_tol=1e-9)` or `np.allclose`. In AI code this shows up as embedding similarity assertions that fail on a different CPU, and as probability sums that are 0.9999999999 instead of 1.0 — so `assert probs.sum() == 1.0` is a landmine. Also beware: `np.float32` and Python `float`

compare with different precision, and reduction order changes results, so the *same* sum computed on GPU and CPU will differ in the last bits.

9.4 Silent broadcasting (see §3)

The only one on this list that produces no exception, no warning, and plausible-looking numbers.

9.5 Pickle is arbitrary code execution

`pickle.load()` on untrusted data runs whatever that data tells it to. This is not a theoretical concern in AI: model checkpoints, cached embeddings, `joblib` artifacts and `.pt` files are pickles, and downloading one from a model hub is running a stranger's code. Use **safetensors** for weights, JSON/Parquet/Arrow for data, and `torch.load(..., weights_only=True)` when you must use torch's loader. Never unpickle a file you did not create. Related: `yaml.load` without `SafeLoader` has the same problem for the same reason.

9.6 The rest of the list

- **is vs ==.** `x is 256` may be True and `x is 257` False, because small ints are interned. Use `is` only for `None`, `True`, `False`.
 - **Shallow copy.** `copy.copy(config)` shares nested dicts. Mutating `config["model"]["params"]` changes the original. `copy.deepcopy` or, better, frozen dataclasses.
 - **Modifying a list while iterating it.** Skips elements, silently. Iterate over a copy or build a new list.
 - **Bare except:** catches `KeyboardInterrupt` and `SystemExit`, so your long-running eval job becomes unkillable. Catch `Exception` at minimum, the specific type ideally.
 - **__pycache__ and stale imports** after you rename a module — if behaviour makes no sense, delete it.
 - **Circular imports** are the tax on a `utils.py` that grew. Restructure; do not paper over it with a function-local import (though that is the emergency fix).
-

10. Where this returns

Idea here	Where it returns
gather + semaphore batching	<code>../../../../08-rag/</code> — embedding thousands of chunks
Backoff, jitter, timeouts	<code>../../../../10-ai-architecture/</code> — resilience patterns
Vectorization, BLAS identity	<code>../../../../02-deep-learning/</code> — this is a forward pass
Broadcasting bugs	<code>../../../../02-deep-learning/</code> — attention-mask shape errors
float32 vs float64	<code>../../../../11-llm-models/</code> — quantization takes this further
Generators over a corpus	<code>../../../../08-rag/</code> — chunking pipelines
Concurrent request batching	<code>../../../../15-ai-evals/</code> — running an eval suite
Pickle / deserialization risk	<code>../../../../13-ai-security/</code> — supply-chain attacks
GIL, event loop, blocking calls	<code>./typescript/</code> [16-ai-tech-stack/tracks/typescript] — where the event loop is the default

Back to 01-modern-python-toolchain.md · On to the lab [\[16-ai-tech-stack/tracks/python/lab/EXERCISES.md\]](#).